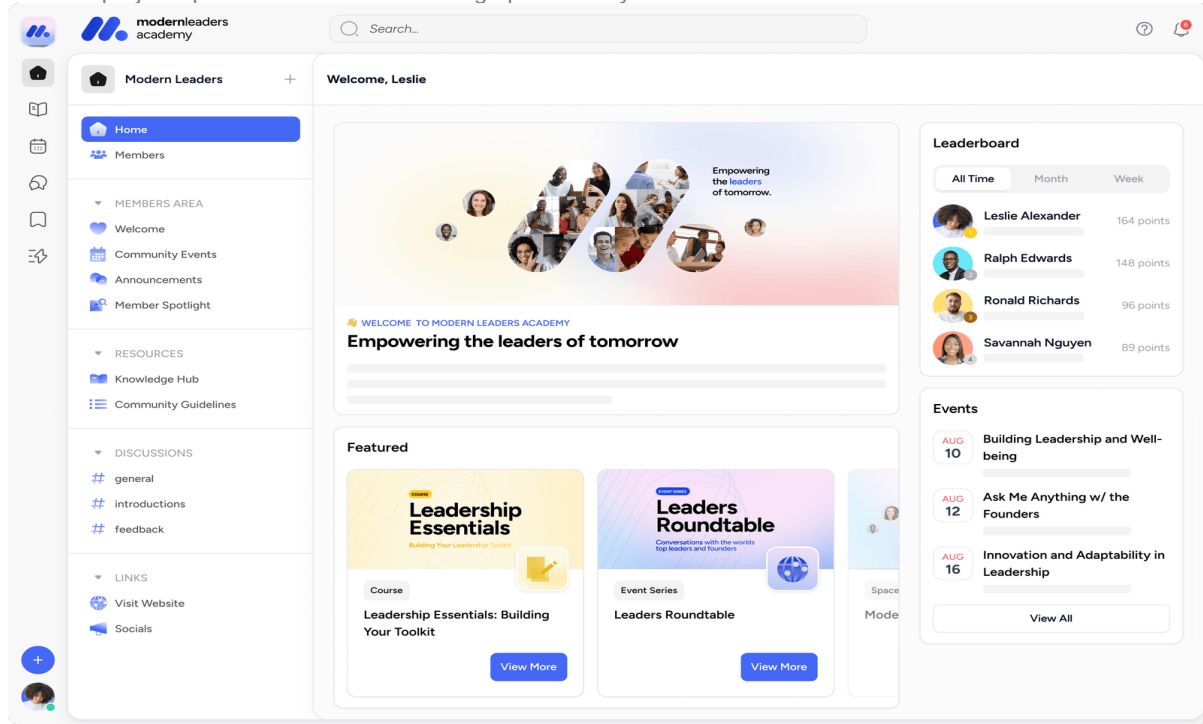


LearnWave — Master Blueprint (Detailed)

Comprehensive technical & design blueprint for LearnWave, an AI-powered academic support platform for VTU students. This document contains full system overview, page-by-page functional flows, role-based routing, Neon (Postgres) schema with Prisma, ER diagram, API routes and payloads, admin portal design, UI mapping to the provided reference, infra and CI/CD, roadmap, and future enhancements.

Generated from project specs and reference UI image provided by the user.



Prepared: Detailed version requested (~40-60 pages) — includes diagrams and code examples.

Table of Contents

1. 1. Executive Summary & Objectives
2. 2. System Overview (Frontend, Backend, Services)
3. 3. Full Page List & Page-by-Page Functional Flow
4. 4. UX/UI Mapping — Exact visual spec and tokens
5. 5. Routing & Role-Based Access Control (Student, Mentor, Admin)
6. 6. Database Design — Neon (Postgres) with Prisma (ER diagram)
7. 7. API Surface, Endpoints & Sample Payloads
8. 8. Frontend Architecture & Component Tree
9. 9. Admin Portal — detailed controls and workflows
10. 10. Security, Privacy, Compliance & Observability
11. 11. Performance, Testing & CI/CD
12. 12. Deployment & Infrastructure
13. 13. Accessibility, Internationalization & QA
14. 14. Implementation Roadmap & Sprint Breakdown
15. 15. Future Enhancements & Appendix (Prisma schema, SQL)

1. Executive Summary & Objectives

LearnWave aims to centralize academic resources, mentorship, and AI-assisted study tools for VTU students. The platform should be polished, secure, and optimized for mobile and desktop with a dashboard-style UX matching the provided reference image. Objectives:

- Deliver an experience similar to the provided dashboard image (colors, spacing, cards, left nav, right side panels).
- Implement role-based features for students, mentors, and admins with secure access controls.
- Use Neon (serverless Postgres) as the single source of truth with Prisma ORM and JSONB for flexible fields.
- Provide a powerful admin portal to moderate, manage users, view analytics, and perform payouts.
- Integrate AI capabilities (summarize, quiz gen) through OpenAI APIs with rate-limited endpoints.

2. System Overview

2.1 Frontend

Recommended stack: Next.js (App Router) with TypeScript, Tailwind CSS, shadcn/ui components, Framer Motion for microinteractions, TanStack Query for remote data, and Zustand for local UI state. Use NextAuth.js for authentication (or a custom JWT strategy if needed). Build component library that mirrors the reference UI precisely.

2.2 Backend

Recommended stack: Node.js with NestJS (or Express + TypeScript). Use Prisma as ORM connected to Neon Postgres. Expose a RESTful API (or GraphQL if team prefers) with strict validation using Zod. Implement RBAC middleware, rate limiting, and logging. Use Stripe or Razorpay for payments.

2.3 Services & Integrations

- Neon Postgres (primary DB) - S3-compatible storage for assets - Redis for caching and job locking - Background jobs: BullMQ (Redis) for email, thumbnailing, virus scanning - Realtime: Socket.IO (or Pusher) for notifications and live sessions - Observability: Sentry, Prometheus, Grafana - AI: OpenAI API for text tasks

3. Full Page List & Page-by-Page Functional Flow

3.1 Page Inventory (exhaustive)

Path	Purpose
/	Landing — marketing, public info
/auth/login	Login
/auth/signup/student	Student Signup (USN, branch, sem)
/auth/signup/mentor	Mentor Signup (resume upload)
/s/dashboard	Student Dashboard (hero, featured, quick links, AI)
/s/resources	Student Resources (filterable)
/s/resources/[id]	Resource viewer (PDF, video)
/s/forum	Forum listing
/s/forum/[thread]	Thread view
/s/timetable	Timetable & calendar
/m/dashboard	Mentor Dashboard
/m/studio	Mentor Studio (create content)
/mentors	Mentor Directory
/mentors/[id]	Mentor profile & booking
/bookings	Bookings list
/admin/dashboard	Admin Dashboard
/admin/users	User management
/admin/approvals	Mentor/resource approvals
/admin/analytics	Analytics & exports
/settings	User settings
/notifications	Notifications center
/help	Help & docs
/legal/privacy	Privacy policy
/legal/terms	Terms of service

3.2 Example: Student Dashboard flow

Landing -> If logged in as student -> /s/dashboard: Hero card with AI quick actions, Featured resources (cards), Left nav for sections, Right panel for leaderboard/events. Widgets are lazy-loaded; resource previews prefetch metadata using TanStack Query.

Interactions: click resource card -> open /s/resources/[id] with inline PDF viewer and AI summary. Click mentor booking -> open modal for time slots and payment. Notifications appear via socket and persist in DB.

3.x Resources & Viewer — Functional Flow

Resources listing supports filters: semester, subject, type. Each item has metadata, tags, related mentors. Viewer supports PDF rendering, video streaming via signed URLs, AI summary, notes, and download. Teachers/mentors can upload versions; version history stored in JSONB metadata.

Prefetch strategy: list uses light metadata; visitor opens viewer and then requests heavy assets (signed URL from S3).

3.x Forum & Discussions — Functional Flow

Thread listing supports pinning, tags, search. Users can upvote, comment, and report. Moderation queues surface flagged posts to admin/mentor reviewers. Use full-text search backed by Postgres tsvector.

3.x Mentor Studio & Booking — Functional Flow

Mentor studio allows creating resource entries, scheduling live sessions, setting prices. Bookings create a pending transaction; payments processed via Stripe/Razorpay. On success, booking status becomes 'confirmed' and calendar invites are sent. Mentor earnings tracked for payouts.

3.x Admin Workflows (approvals, moderation) — Functional Flow

Admin approvals: mentors submit profile + resume. Admin reviews resume, approves or requests changes. Resource moderation allows editing metadata and taking down inappropriate content. Admin can impersonate any user for debugging with audit log entry.

4. UX/UI Mapping — Visual Spec & Tokens

4.1 Color Palette (derived from reference image)

Name	Hex
Primary Blue	#4B7CFF
Accent Gradient 1	#FFF7E6 (cream)
Accent Gradient 2	#FFDDE6 (pink)
Accent Gradient 3	#E8F0FF (soft blue)
Text Primary	#0F172A
Text Muted	#475569
Card BG	#FFFFFF (with subtle shadow)

4.2 Typography

- Primary: Inter (variable) for UI. - Sizes: H1 28px, H2 20px, Body 14px, Small 12px. - We recommend using system font fallback stack with Inter as variable.

4.3 Spacing and Layout Tokens (Tailwind)

Tailwind theme additions: primary color, radii: rounded-2xl for cards, shadows: soft large, and container widths to match image.

Example Tailwind tokens (tailwind.config.js):

```
module.exports = { theme: { extend: { colors: { primary: '#4B7CFF', accent1: '#FFF7E6' /* etc */ }, borderRadius: { '2xl': '1rem' }, boxShadow: { 'soft': '0 8px 20px rgba(15,23,42,0.06)' } } } }
```

5. Routing & Role-Based Access Control (RBAC)

5.1 Next.js App Router structure

Use folder grouping to separate role-scoped areas and apply middleware for role enforcement. Example structure:

```
/app /(public)/ /(auth)/ /(student)/s/dashboard/page.tsx /(mentor)/m/dashboard/page.tsx
/(admin)/admin/dashboard/page.tsx
```

5.2 Middleware & Enforcement

Implement server-side middleware that inspects the session cookie/JWT, fetches role from DB or session, and enforces access. Example pseudo-code:

```
export async function middleware(req) { const token = getToken(req); // JWT or NextAuth if
(!token) return NextResponse.redirect('/auth/login'); const role = token.role; if
(req.nextUrl.pathname.startsWith('/admin') && role !== 'admin') return
NextResponse.redirect('/403'); if (req.nextUrl.pathname.startsWith('/m') && role !== 'mentor')
return NextResponse.redirect('/403'); // allow return NextResponse.next(); }
```

6. Database Design — Neon (Postgres)

6.1 Core Tables & Relationships (summary)

Table	Columns
users	id, email, password_hash, role, name, avatar_url, created_at
students	user_id FK, usn, branch, semester, profile_json
mentors	user_id FK, resume_url, approved, rating_avg
resources	id, title, type, author_id, visibility, subject, semester, metadata jsonb
forum_threads	id, title, author_id, tags, tsvector
posts	id, thread_id, author_id, content, parent_id
bookings	id, student_id, mentor_id, start, end, status, amount, payment_id
notifications	id, user_id, type, data jsonb, read boolean, created_at
audit_logs	id, user_id, action, meta jsonb, created_at

6.2 ER Diagram (vector drawing)

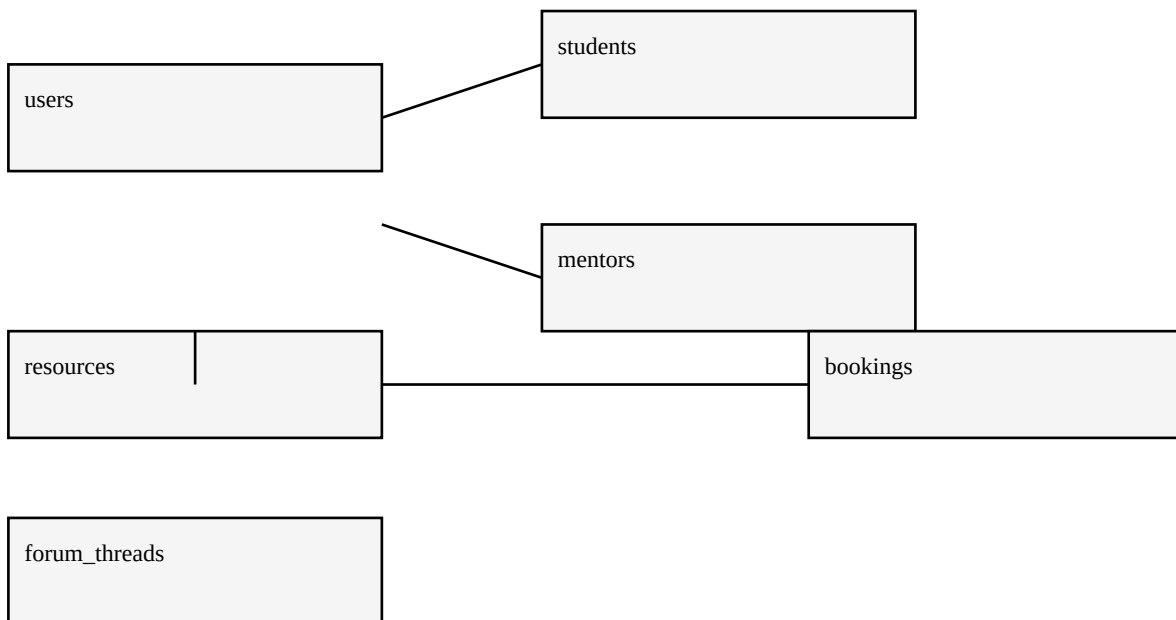


Diagram: users -> students (1:1), users -> mentors (1:1), resources authored by users, bookings link students and mentors, forum threads and posts link to users.

7. API Surface & Sample Payloads

7.1 Auth

POST /api/auth/login — payload: { email, password } — returns { token, user }

POST /api/auth/login Request: { "email": "student@vtu.edu", "password": "....." } Response: { "token": "jwt...", "user": { "id":"cuid()", "email":"student@vtu.edu", "role":"student" } }

7.2 Users & Admin

GET /api/admin/users — paginated list (admin only). PATCH /api/users/:id — update profile (admin or owner).

PATCH /api/users/:id Request body: { "name": "New Name", "avatarUrl": "https://..." }

7.3 Resources

GET /api/resources?semester=4&type=pdf — returns list with metadata. POST /api/resources — create resource (mentor/admin).

POST /api/resources { "title":"Leadership Essentials", "type":"pdf", "subject":"Management", "semester":4, "fileKey":"s3://bucket/key.pdf" }

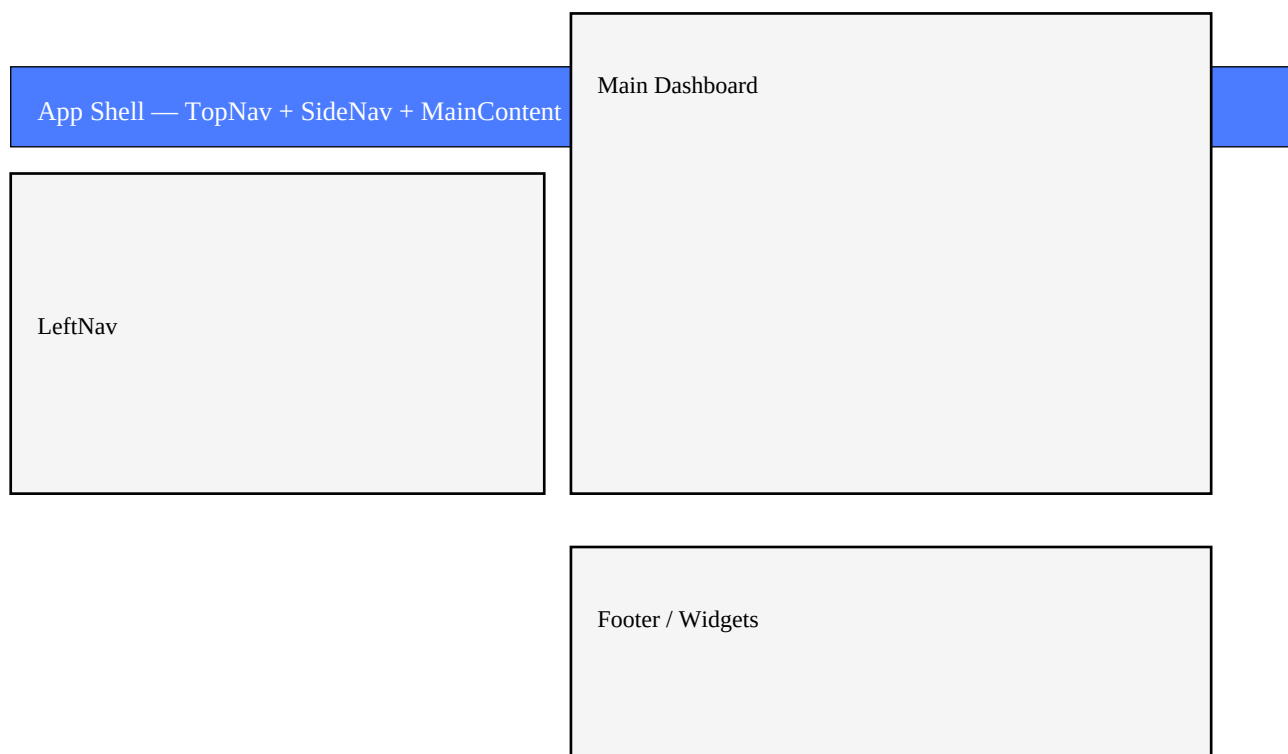
7.4 Bookings & Payments

POST /api/bookings — creates booking, returns paymentIntent client secret (Stripe). Webhook /api/webhooks/payments handles status updates.

POST /api/bookings { "mentorId":"mid_123", "studentId":"sid_123", "start":"2025-11-18T10:00:00Z", "end":"2025-11-18T11:00:00Z", "amount": 5000 }

8. Frontend Architecture & Component Tree

8.1 Component Tree (visual)



Key components and responsibilities: TopNav (search, avatar, notifications). SideNav (sections and quick actions). HeroCard, FeaturedCard, LeaderboardPanel, EventsPanel, ResourceViewer, ForumThread, MentorStudio editor.

8.2 State Management & Data Fetching

Use TanStack Query for server state with caching & optimistic updates. Use Zustand (or Jotai) for ephemeral UI state such as open modals, theme toggle. Prefetching strategy for resources to reduce perceived latency. Use React Query Devtools during development.

9. Admin Portal — Capabilities & UX

9.1 Immediate Post-login Admin Dashboard

Admin dashboard shows pending approvals, recent activity, key metrics (DAU, MAU), revenue, and moderation queue. Quick search and global filters, bulk actions for content removal or user suspension.

9.2 Mentor Approval workflow

Mentor submits profile -> admin reviews resume and portfolio -> admin approves or requests changes. Approval triggers notification and makes mentor visible in directory. All actions logged in audit_logs.

9.3 Impersonation & Safety

Admin can impersonate a user for debugging; every impersonation is logged with admin id, target id, timestamp, and reason. Provide a 'stop impersonation' button returning admin to their account.

10. Security & Compliance

10.1 Authentication & Authorization

NextAuth or custom JWT. Passwords hashed with bcrypt/argon2. RBAC enforced in middleware & service layer. Sessions short-lived; use refresh tokens securely stored.

10.2 Data Protection & Privacy

Encrypt sensitive PII at rest. Provide endpoints for data export and deletion to comply with GDPR. Keep audit logs immutable for at least 90 days.

10.3 Observability & Logging

Sentry for error tracking. Prometheus metrics + Grafana dashboards for system health. Centralized logs via Loki or Cloud provider logging. Retention policy per compliance rules.

11. Performance & CI/CD

11.1 Caching & CDN

Use CDN for static assets and signed URLs for media behind CDN. Redis for caching heavy DB queries and session caching if needed. Use Next.js ISR/SSG for public content where appropriate.

11.2 Testing Strategy

- Unit tests: Vitest/Jest for components & services - Integration: Playwright for E2E - Load testing: k6 or Locust for critical paths (login, resource download, booking flow) - Security scanning: Snyk or Dependabot

11.3 CI/CD Pipeline (example)

GitHub Actions pipeline: lint -> unit tests -> build -> deploy to staging -> run integration tests -> deploy to production after approvals. Add canary releases for backend API.

12. Deployment & Infrastructure

12.1 Neon Postgres considerations

Use Neon for serverless Postgres. Ensure connection pooling, use Prisma Data Proxy recommended pattern or a pooling solution. Use read replicas for analytics queries against large datasets.

12.2 Hosting & Storage

- Frontend: Vercel (or Netlify) - Backend: Render / Fly / AWS ECS Fargate - Storage: AWS S3 (or Wasabi) with signed URLs - Redis: Managed Redis (Upstash/Redis Cloud) - Secrets: use platform secrets or HashiCorp Vault

13. Accessibility & i18n

Ensure WCAG AA compliance: color contrast checks, keyboard navigation, focus states, screen reader labels. Provide locale fallback and translation JSONs using Next.js i18n or i18next.

13.1 QA Checklist

- Keyboard-only navigation tested - Screen reader tests (NVDA/VoiceOver) - Contrast and color blindness checks - Performance budgets enforced - Regression tests in CI

14. Implementation Roadmap & Sprint Breakdown

Proposed 8-week plan with deliverables per week. Each week contains tasks, owners, and acceptance criteria. Below is an expanded sprint plan.

Sprint	Focus
Week 1	Auth, RBAC, DB schema, UI shell, basic nav
Week 2	Student dashboard, resources listing, file upload & S3 integration
Week 3	Mentor signup flow, mentor approvals, basic forum
Week 4	Bookings and payments integration, mentor studio MVP
Week 5	AI integrations: summarize, quizgen; resource viewer enhancements
Week 6	Admin analytics, audit logs, moderation tools
Week 7	Performance tuning, caching, accessibility fixes
Week 8	QA, load testing, launch prep, documentation

15. Future Enhancements

- Mobile native apps (React Native or Expo) - Offline-first reading mode for resources - AI tutor with conversational interface and session history - Advanced analytics with cohort retention visualization - Multi-tenant support for different universities

Appendix A: Full Prisma schema (expanded)

```
generator client { provider = "prisma-client-js" } datasource db { provider = "postgresql" url = env("DATABASE_URL") } model User { id String @id @default(cuid()) email String @unique password String name String? role Role @default(STUDENT) avatarUrl String? createdAt DateTime @default(now()) student Student? mentor Mentor? resources Resource[] posts Post[] threads ForumThread[] bookingsAsStudent Booking[] @relation("StudentBookings") bookingsAsMentor Booking[] @relation("MentorBookings") } model Student { id String @id @default(cuid()) user User @relation(fields: [userId], references: [id]) userId String @unique usn String branch String semester Int profile Json? } model Mentor { id String @id @default(cuid()) user User @relation(fields: [userId], references: [id]) userId String @unique resumeUrl String? bio String? approved Boolean @default(false) ratingAvg Float? } model Resource { id String @id @default(cuid()) title String type String authorId String author User @relation(fields: [authorId], references: [id]) subject String? semester Int? metadata Json? createdAt DateTime @default(now()) } model ForumThread { id String @id @default(cuid()) title String authorId String author User @relation(fields: [authorId], references: [id]) tags String[] tsv String? // for tsvector (managed outside prisma) } model Post { id String @id @default(cuid()) threadId String authorId String content String parentId String? createdAt DateTime @default(now()) author User @relation(fields: [authorId], references: [id]) } model Booking { id String @id @default(cuid()) studentId String mentorId String start DateTime end DateTime status String amount Int paymentId String? student User @relation("StudentBookings", fields: [studentId], references: [id]) mentor User @relation("MentorBookings", fields: [mentorId], references: [id]) } enum Role { STUDENT MENTOR ADMIN }
```

Appendix B: Sample SQL Indexes & Full-Text Search

Example: GIN index for JSONB and tsvector for resources and forum threads:

```
-- resources full-text index ALTER TABLE resources ADD COLUMN search_tsv tsvector; UPDATE resources SET search_tsv = to_tsvector('english', coalesce(title, '') || ' ' || coalesce(metadata->>'description', '')); CREATE INDEX resources_search_idx ON resources USING GIN (search_tsv); -- JSONB GIN index CREATE INDEX resources_metadata_gin ON resources USING GIN (metadata);
```

Detailed Section 1: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 2: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 3: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 4: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 5: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 6: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 7: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 8: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 9: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 10: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 11: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 12: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 13: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 14: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 15: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 16: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 17: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 18: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 19: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 20: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 21: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 22: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 23: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 24: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 25: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 26: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 27: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 28: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 29: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

Detailed Section 30: Deep-dive

This section contains deep-dive technical notes, implementation details, and example snippets for engineers and designers. It can be used to hand off to development teams.

Architecture Patterns & Best Practices

- Use hexagonal architecture for backend services.
- Keep domain logic in services, controllers thin.
- Use feature flags for gradual rollouts.
- Document API with OpenAPI/Swagger and keep it versioned.

Data Migration & Backfills

Plan migration scripts using Prisma migrations; for big datasets use background jobs and chunked updates. Maintain idempotent migration scripts.

Testing & Acceptance Criteria

Each feature must have unit tests, contract tests, and E2E scenarios. Acceptance criteria example for resource upload: successful upload returns signed URL, DB row created with metadata, file accessible via signed URL for 7 days.

References & Files Used

Source specification files provided by user and the reference UI image were used to craft this blueprint.

Files included in original upload: LearnWave_Spec.pdf, ■ LearnWave.docx, reference dashboard image.